Advanced Sorting

DSA27 — Lecture 10 · Data Structures and Algorithms

Dr. Haitham A. El-Ghareeb

Faculty of Computers and Information Sciences, Mansoura University

Week 10 · Fall 2026

Where We Are

From n^2 to $n \log n$

Week 9: bubble, selection, insertion — $O(n^2)$ comparisons.

n	$n^2/2$	$n \log_2 n$
1,000	500,000	10,000
1,000,000	500,000,000,000	20,000,000

Today: three sorts on the right-hand column — and the price each one pays.

Today

1. Divide and conquer
2. Merge sort: $T(n) = 2T(n/2) + n$, stable, $O(n)$ extra space
3. Quicksort: partition, pivots, and a worst case you will **measure**
4. Heap sort: the array is the tree — Week 12 arriving early
5. Measured: all six sorts, one picture
6. What Python's `sorted()` actually does
7. Stability, and the table to remember

Divide and Conquer

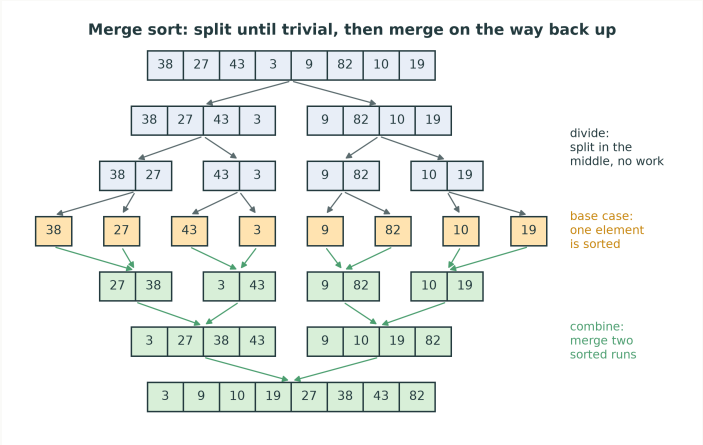
Three steps

1. **Divide** the problem into smaller problems of the same kind.
2. **Conquer** each one — recursively, until it is trivial.
3. **Combine** their answers into the answer.

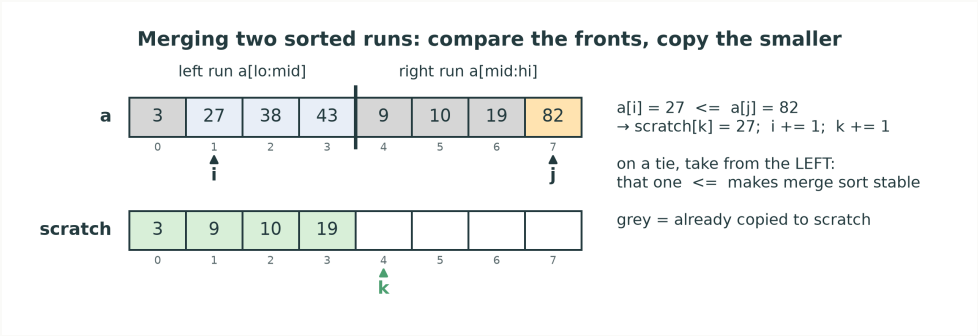
Merge sort: divide is free, **combine** does the work. Quicksort: **divide** does the work, combine is free.

Merge Sort

Split until trivial, merge on the way up



Merging two sorted runs



Compare the two **fronts**; copy the smaller to `scratch`; advance that side. When one run is empty, copy the rest of the other.

_merge: the code

```
def _merge(a, lo, mid, hi, scratch):
    i, j, k = lo, mid, lo
    while i < mid and j < hi:
        if a[i] <= a[j]:
            scratch[k] = a[i]
            i += 1
        else:
            scratch[k] = a[j]
            j += 1
        k += 1
    while i < mid:
        scratch[k] = a[i]
        i += 1
        k += 1
    while j < hi:
        scratch[k] = a[j]
        j += 1
        k += 1
    for k in range(lo, hi):
        a[k] = scratch[k]
```

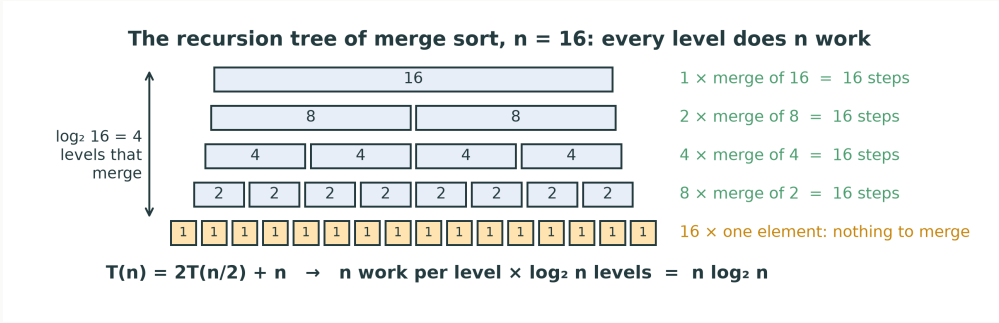
merge_sort_steps: divide, conquer, combine

```
a = _copy(values)
scratch = Array(len(a))
yield snapshot(a), ()

def sort(lo, hi):
    if hi - lo <= 1:
        return
    mid = (lo + hi) // 2
    yield from sort(lo, mid)
    yield from sort(mid, hi)
    _merge(a, lo, mid, hi, scratch)
    yield snapshot(a), tuple(range(lo, hi))

yield from sort(0, len(a))
yield snapshot(a), ()
```

How much work? $T(n) = 2T(n/2) + n$



$\log_2 n$ levels, n work per level: $O(n \log n)$ — best, average and worst case alike.

The price: $O(n)$ extra space

- Merge sort is **not in place**: `scratch` holds n slots.
- The call stack adds $O(\log n)$.
- Fine for n = 10^6 numbers; a real cost on a small device, or for data that barely fits in memory.

Quicksort

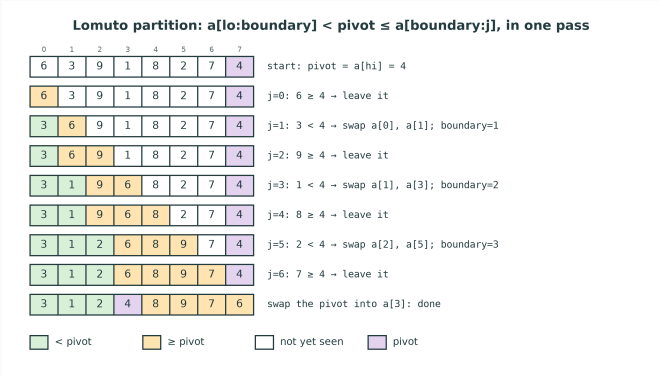
Partition, then recurse

Pick a **pivot**. Rearrange so that

$$\underbrace{< \text{pivot}}_{\text{left part}} \quad \text{pivot} \quad \underbrace{\geq \text{pivot}}_{\text{right part}}$$

The pivot is now in its **final place**. Sort the two parts; nothing to combine.

Lomuto’s partition



_partition: the code

```
def _partition(a, lo, hi):
    pivot = a[hi]
    boundary = lo
    for j in range(lo, hi):
        if a[j] < pivot:
            a[boundary], a[j] = a[j], a[boundary]
            boundary += 1
    a[boundary], a[hi] = a[hi], a[boundary]
    return boundary
```

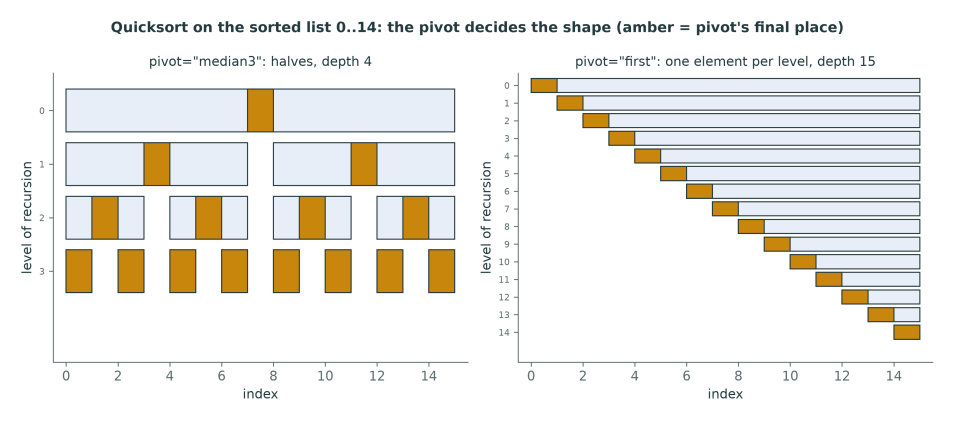
Invariant: `a[lo:boundary] < pivot <= a[boundary:j]`.

Choosing the pivot

```
def _choose_pivot(a, lo, hi, strategy, rng):
    if strategy == "first":
        return lo
    if strategy == "last":
        return hi
    if strategy == "random":
        return rng.randint(lo, hi)
    if strategy == "median3":
        mid = (lo + hi) // 2
        first, middle, last = a[lo], a[mid], a[hi]
        if first <= middle <= last or last <= middle <= first:
            return mid
        if middle <= first <= last or last <= first <= middle:
            return lo
        return hi
    raise ValueError(...)
```

The chosen pivot is swapped to `a[hi]`; then `_partition` runs as before.

The worst case, measured



Sorted input, `pivot="first"`: every partition peels off **one** element.
$(n-1)+(n-2)+\cdots+1 = n(n-1)/2$ comparisons: $O(n^2)$.

Why the average is $O(n \log n)$

- A pivot anywhere in the **middle half** splits no worse than 1/4 : 3/4.
- Then the largest part shrinks by 3/4 per level: about $\log_{4/3} n$ levels of $O(n)$ work.
- Half of all pivots are in the middle half. **Random pivots: expected** $O(n \log n)$, about $1.39\, n \log_2 n$ comparisons.

Stack depth: recurse on the smaller side

```
def sort(lo, hi):
    while lo < hi:
        p = _choose_pivot(a, lo, hi, pivot, rng)
        a[p], a[hi] = a[hi], a[p]
        q = _partition(a, lo, hi)
        yield snapshot(a), (q,)
        if q - lo < hi - q:
            yield from sort(lo, q - 1)
            lo = q + 1
        else:
            yield from sort(q + 1, hi)
            hi = q - 1
```

The **smaller** part is a call; the **larger** part is the next turn of the loop. Stack depth: $O(\log n)$, even in the worst case.

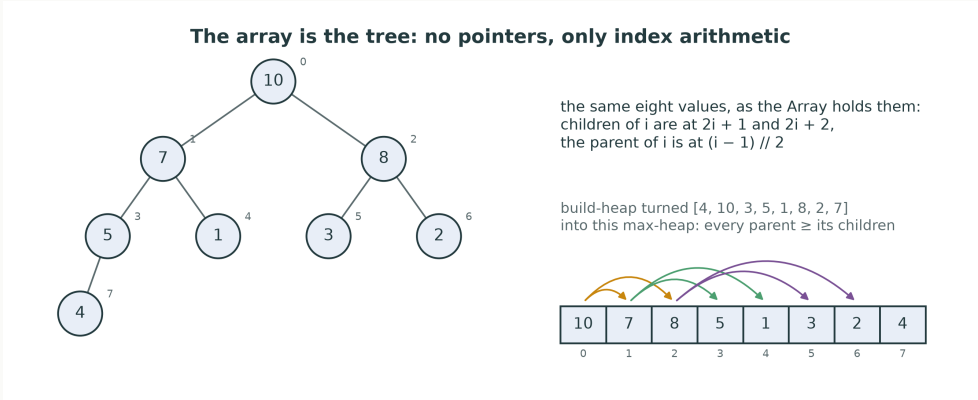
Quicksort is not stable

The partition swaps across long distances: an element can jump over its equal.

`quick_sort` on `[3a, 1b, 3c, 2d, 1e, 2f]` (sorted by number) gives `[1b, 1e, 2d, 2f, 3c, 3a]` — **3c before 3a**.

Heap Sort

The array is the tree



A **max-heap**: every parent $\geq$ its children. So the maximum is at `a[0]`.

Sift down: repair one parent

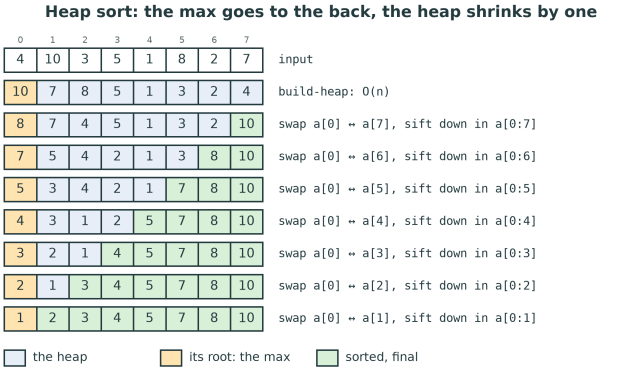
```
def _sift_down(a, index, size):
    while True:
        largest = index
        left, right = 2 * index + 1, 2 * index + 2
        if left < size and a[left] > a[largest]:
            largest = left
        if right < size and a[right] > a[largest]:
            largest = right
        if largest == index:
            return
        a[index], a[largest] = a[largest], a[index]
        index = largest
```

Build-heap in $O(n)$, then sort

```
for index in range(n // 2 - 1, -1, -1):        # build the heap: O(n)
    _sift_down(a, index, n)
for end in range(n - 1, 0, -1):
    a[0], a[end] = a[end], a[0]                 # the max goes to the back
    _sift_down(a, 0, end)                       # restore the heap in a[:end]
```

1. **Build** a max-heap: sift down every parent, last to first.
2. **Sort**: swap the max to the back; the heap shrinks by one; sift down.

Heap sort, traced

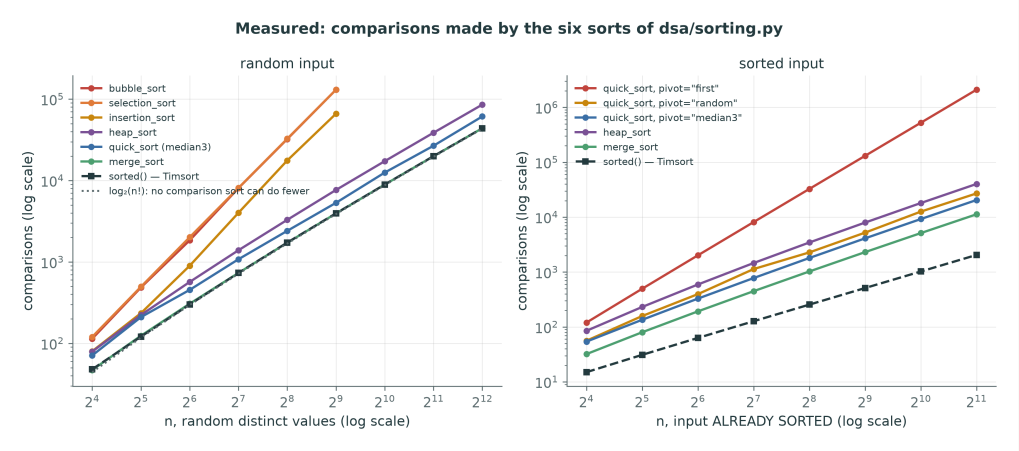


Heap sort: the trade-offs

- $O(n \log n)$ **always** — no bad input, like merge sort.
- $O(1)$ **extra space** — in place, like quicksort; no recursion at all.
- **Not stable**: the swap to the back jumps over equals.
- Slower in practice: it jumps around memory ($i \to 2i + 1$).

Measured

Six sorts, one picture



Can anything beat $n \log n$?

A comparison sort must tell apart all **n!** orders of its input. Each comparison has two outcomes, so it needs at least $\log_2(n!) \approx n \log_2 n - 1.44\, n$ comparisons in the worst case.

Merge sort and heap sort are optimal — up to a constant.

What Python Actually Uses

`sorted()` and `list.sort()`: Timsort

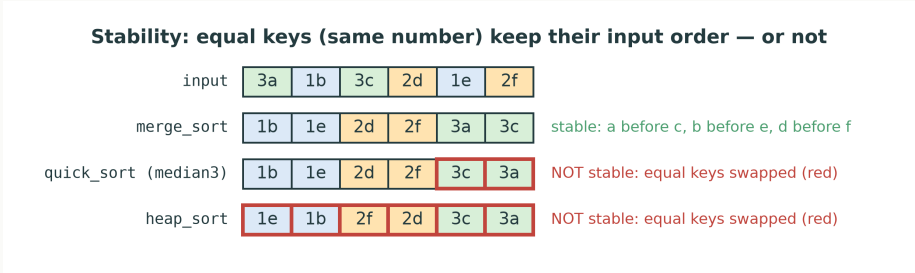
Enrichment — not examinable.

1. Find the **runs** already in the data (reversing descending ones).
2. Extend short runs to 32–64 elements with **insertion sort**.
3. **Merge** the runs, like merge sort — stable, $O(n)$ extra space.

Sorted input: one run, **n – 1 comparisons**. Worst case: $O(n \log n)$.

Stability

Equal keys, input order



A sort is **stable** if equal keys keep their input order.

The table to remember

Sort	Best	Average	Worst	Extra space	Stable
bubble	n	n^2	n^2	$O(1)$	yes
selection	n^2	n^2	n^2	$O(1)$	no
insertion	n	n^2	n^2	$O(1)$	yes
merge	$n \log n$	$n \log n$	$n \log n$	$O(n)$	**yes**
quick	$n \log n$	$n \log n$	n^2	$O(\log n)$	no
heap	$n \log n$	$n \log n$	$n \log n$	$O(1)$	no
counting	$n + k$	$n + k$	$n + k$	$O(n + k)$	—
Timsort	n	$n \log n$	$n \log n$	$O(n)$	yes

This Week

Exercises: `dsa/sorting.py`, second half

Function	The trap
`merge_sort_steps`	`yield from` the recursive calls; `<=` in the merge; one scratch `Array`
`quick_sort_steps`	Lomuto's invariant; swap the pivot to `hi`; smaller side first
`heap_sort_steps`	build from `n // 2 - 1` **down** to 0; sift with `size = end`

```
pytest tests/test_sorting.py -v -k "merge or quick or heap"
```

Homework 10 — before Lecture 11

1. **Implement** merge, quick and heap sort until the 37 tests pass.
2. **Trace** `_partition` on `[5, 8, 1, 9, 3, 7, 2, 6]` as a table of `j`, `a[j]`, `boundary` and the array.
3. **Draw** the merge tree of `merge_sort([6, 5, 3, 1, 8, 7, 2, 4])`.
4. **Count** comparisons for `quick_sort(list(range(n)), pivot)` with `"first"` and `"median3"`, n = 100, 200, 400. Predict first.
5. **Break it:** change <= to < in `_merge`. Which tests fail? What else changes?

Practise online

- **Week 10 quiz**: 22 multiple-choice questions, every answer explained
- **Written practice** on the same page: 5 questions. Write, then mark yourself against the marking points
- In Arabic: week 10

Summary

Seven things to keep

1. **Divide and conquer:** split, solve the parts recursively, combine.
2. **Merge sort:** $T(n) = 2T(n/2) + n = O(n \log n)$ always; $O(n)$ extra; stable because of one `<=`.
3. **Quicksort:** partition around a pivot; the pivot lands in its final place.
4. `pivot="first"` on sorted input: $n(n-1)/2$ comparisons — **measured**. Random or median-of-three: $O(n \log n)$ on average.
5. Recurse on the **smaller** part: $O(\log n)$ stack, always.
6. **Heap sort:** the array is the tree; build-heap $O(n)$; $O(n \log n)$ always, $O(1)$ extra, not stable.
7. No comparison sort beats $\log_2(n!) \approx n \log_2 n$. Python's `sorted()` is **Timsort**: merge sort that exploits existing runs.

Next

Week 11 — Trees. Binary search trees: the sorted array's fast search, without its $O(n)$ insert. And an in-order traversal gives the values **sorted** — one more sort, for free.